

Build: AI Narrative Layer

Objective

Create an **AI Narrative Layer** that converts deterministic EnterprateAI outputs into clear, business-friendly explanations.

It must explain:

- business health
- engine scores
- risks
- recommendations
- fragility
- scenario outcomes
- adaptive intelligence decisions
- PDF report sections

Important rule:

The AI Narrative Layer must explain deterministic outputs. It must not calculate, invent, or override scores.

Recommended Folder

`/src/lib/narratives`

Suggested structure:

```
/src/lib/narratives
├─ types.ts
├─ prompts.ts
├─ narrative-generator.ts
├─ narrative-validator.ts
```

```
|— narrative-generator.test.ts  
|— index.ts
```

1. Input Contract

```
export interface NarrativeInput {  
  businessId: string;  
  
  output: EnterpriseIntelligenceOutput;  
  
  narrativeType:  
    | "executive_summary"  
    | "risk_explanation"  
    | "recommendation_summary"  
    | "scenario_explanation"  
    | "fragility_explanation"  
    | "pdf_report_section"  
    | "advisor_chat";  
  
  audience?: "founder" | "advisor" | "investor" | "internal";  
  
  tone?: "simple" | "professional" | "strategic";  
  
  metadata?: {  
    model?: string;  
    promptVersion?: string;  
    maxWords?: number;  
  };  
}
```

2. Output Contract

```
export interface NarrativeOutput {  
  businessId: string;
```

```
narrativeType: string;

headline: string;

narrative: string;

keyPoints: string[];

actionSummary: string[];

confidenceNote: string;

sourceReferences: {
  scoresUsed: string[];
  risksUsed: string[];
  recommendationsUsed: string[];
  scenarioUsed?: string;
};

validation: {
  status: "passed" | "failed";
  issues: string[];
};

trace: {
  promptVersion: string;
  model?: string;
  generatedAt: string;
};
}
```

3. Grounding Rules

The AI must only use data from:

EnterpriseIntelligenceOutput

Allowed sources:

- executiveSummary
- scores
- classifications
- engineResults
- fragility
- risks
- recommendations
- scenarios
- adaptiveIntelligence
- confidence

The AI must not invent:

- new scores
- new percentages
- fake risks
- unsupported recommendations
- unverified market claims
- financial projections not present in the output

4. Narrative Types

A. Executive Summary

Purpose:

Explain the overall business condition.

Output should answer:

- What is happening?
- What is the strongest area?
- What is the weakest area?
- What should the business prioritise?

Example:

The business shows strong growth potential, but it is currently exposed by limited cash resilience. Before scaling further, the priority should be to improve runway and reduce unnecessary fixed costs.

B. Risk Explanation

Purpose:

Explain top risks in plain business language.

Example:

The main risk is financial survival. The business has limited runway, which means it may struggle to absorb delays in revenue or unexpected costs.

C. Recommendation Summary

Purpose:

Turn recommendations into action-focused advice.

Example:

The immediate priority is to reduce non-essential fixed costs and improve short-term cash inflows. This should be addressed before taking on new fixed commitments.

D. Scenario Explanation

Purpose:

Explain simulation outcomes.

Example:

The scenario improves viability but increases fragility. This means the change may look commercially attractive, but it could place additional pressure on operations or cashflow.

E. Fragility Explanation

Purpose:

Explain structural vulnerability.

Example:

The business is fragile because it depends heavily on a small number of customers and has limited operational backup. If one key customer leaves, the effect could spread into revenue and cashflow pressure.

F. PDF Report Section

Purpose:

Generate formal report-ready sections.

Must be:

- professional
- structured
- concise
- suitable for export

G. Advisor Chat

Purpose:

Answer user questions using only the deterministic output.

Example:

User asks:

Why is my score low?

AI answers using:

- weakest engine
- top risks
- recommendations
- confidence notes

5. Prompt Template

```
export const BASE_NARRATIVE_PROMPT = `
You are the EnterprateAI Narrative Engine.
```

```
Your job is to explain deterministic business intelligence outputs in
clear, professional business language.
```

```
Rules:
```

1. Do not invent scores, metrics, risks, recommendations, or financial projections.
2. Use only the provided structured intelligence output.
3. Do not contradict the deterministic engine outputs.
4. If confidence is LOW, clearly state that the insight is based on limited data.
5. Be concise, practical, and business-friendly.
6. Focus on what the business owner should understand and do next.

```
Audience: {{audience}}
```

```
Tone: {{tone}}
```

```
Narrative Type: {{narrativeType}}
```

```
Structured Intelligence Output:
```

```
{{jsonOutput}}
```

```
`;
```

6. Validation Layer

Create `narrative-validator.ts`.

The validator should check:

A. Score Consistency

If narrative mentions a score, it must exist in `output.scores`.

Reject if it contains unsupported numbers.

B. Risk Consistency

If narrative mentions a risk, it must exist in:

`output.risks.topRisks`

or:

`output.fragility.topFragilityFactors`

C. Recommendation Consistency

If narrative gives an action, it should align with:

`output.recommendations.topRecommendations`

D. Confidence Disclosure

If overall confidence is LOW, narrative must include a cautionary note.

7. Narrative Generator Logic

```
export async function generateNarrative(  
  input: NarrativeInput,  
  llmClient: LLMClient  
) : Promise<NarrativeOutput> {  
  const prompt = buildPrompt(input);  
  
  const rawNarrative = await llmClient.generate(prompt);  
  
  const structuredOutput = parseNarrative(rawNarrative);  
  
  const validation = validateNarrative(structuredOutput,  
    input.output);  
  
  return {  
    ...structuredOutput,  
    validation,  
    trace: {  
      promptVersion: input.metadata?.promptVersion ?? "v1.0",  
      model: input.metadata?.model,  
      generatedAt: new Date().toISOString()  
    }  
  };  
}
```

8. Recommended Safer Approach for MVP

For MVP, use **template-first narratives**, then optionally enhance with LLM.

This reduces hallucination risk.

Template-first example

```
function generateExecutiveSummaryTemplate(output:
EnterpriseIntelligenceOutput): NarrativeOutput {
  return {
    headline: output.executiveSummary.headline,
    narrative: output.executiveSummary.summary,
    keyPoints: [
      `Business Intelligence Score:
${output.scores.businessIntelligenceScore}`,
      `Fragility Index: ${output.scores.fragilityIndex}`,
      `Primary classification: ${output.classifications.primary}`
    ],
    actionSummary: output.executiveSummary.nextBestActions,
    confidenceNote: output.confidence.dataQualityNotes.join(" "),
    ...
  };
}
```

Recommended MVP logic:

```
Deterministic template
↓
Optional LLM polish
↓
Validator
↓
Saved narrative
```

9. Database Persistence

Save generated narratives into:

```
CREATE TABLE ai_narratives (
  id UUID PRIMARY KEY,
  business_id UUID NOT NULL,
  source_type VARCHAR NOT NULL,
```

```
source_id UUID,  
narrative_type VARCHAR NOT NULL,  
prompt_version VARCHAR,  
model_name VARCHAR,  
input_context_snapshot JSONB,  
generated_text TEXT,  
validation_status VARCHAR,  
confidence_level VARCHAR,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

10. API Endpoints

Generate Narrative

POST /businesses/:id/narratives

Body:

```
{  
  "sourceType": "assessment",  
  "sourceId": "assessment_uuid",  
  "narrativeType": "executive_summary",  
  "audience": "founder",  
  "tone": "professional"  
}
```

Response:

```
{  
  "success": true,  
  "narrativeId": "uuid",  
  "output": {  
    "headline": "...",  
    "narrative": "...",  
    "keyPoints": [],  
    "actionSummary": [],  
    "confidenceNote": "..."  
  }  
}
```

```
}  
}
```

11. Unit Tests

Create tests for:

- executive summary narrative
- risk explanation narrative
- recommendation narrative
- fragility narrative
- scenario narrative
- low confidence disclosure
- unsupported score rejection
- unsupported risk rejection

12. Acceptance Criteria

Codex should complete this when:

- NarrativeInput and NarrativeOutput exist
- prompt builder exists
- template-first narrative generator exists
- optional LLM generator is supported
- validation layer checks unsupported claims
- low-confidence narratives include caution
- narratives can be saved to database
- API endpoint can generate narratives
- unit tests pass

13. Codex Prompt

Build the EnterpriseAI AI Narrative Layer in `/src/lib/narratives`.

The AI Narrative Layer must convert deterministic `EnterpriseIntelligenceOutput` into business-friendly explanations.

Important:

- It must not calculate scores.
- It must not invent risks.
- It must not invent recommendations.
- It must use only the provided structured output.
- It should use template-first narratives for MVP, with optional LLM polish.
- All LLM outputs must pass validation.

Create:

- `types.ts`
- `prompts.ts`
- `narrative-generator.ts`
- `narrative-validator.ts`
- `narrative-generator.test.ts`
- `index.ts`

Implement narrative types:

- `executive_summary`
- `risk_explanation`
- `recommendation_summary`
- `scenario_explanation`
- `fragility_explanation`
- `pdf_report_section`
- `advisor_chat`

Add:

- `NarrativeInput`
- `NarrativeOutput`
- `prompt builder`
- `template narrative generator`

- optional llmClient interface
- validator for unsupported scores, unsupported risks, unsupported recommendations
- low confidence disclosure logic

Also create persistence support for ai_narratives table and API endpoint:

POST /businesses/:id/narratives

Add unit tests for:

- executive summary
- risk explanation
- recommendation summary
- scenario explanation
- fragility explanation
- low confidence disclosure
- unsupported score rejection
- unsupported risk rejection

Export all narrative utilities from index.ts.